

App Dev Project Report

1. Student Details

Name: Arfat Islam Ansari

Roll Number: 23f2001163

Email: 23f2001163@ds.study.iitm.ac.in

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

Institutes require an efficient system to manage campus recruitment activities involving students, companies, and placement drives. In many institutes, placement activities are still managed using spreadsheets, emails, or manual coordination. This makes it difficult to manage company registrations, track student applications, maintain placement history, and avoid duplicate applications.

The objective of this project is to design and develop a **web-based Placement Portal Application** that allows three types of users — **Admin, Company, and Student** — to interact with the system according to their roles. The system enables companies to create placement drives, students to apply for jobs, and the admin to manage and monitor the entire placement process.

The application ensures proper management of recruitment activities such as company approval, drive approval, application tracking, and placement history while maintaining data consistency and preventing duplicate applications.

Placement Portal Application

Approach:

The Placement Portal application was developed using **Flask** as the backend framework, **SQLite** as the database, and **HTML/CSS (Bootstrap) with Jinja2 templating** for the frontend, strictly following the project requirements

The system consists of three main user roles:

Admin:

The admin acts as the placement cell and is the pre-defined superuser of the system. The admin has full control over the platform, including approving or rejecting company registrations and placement drives. The admin can view and manage all students, companies, drives, and applications, as well as search and monitor system data. This role ensures that only valid

companies and approved drives are visible in the system and maintains overall integrity of the placement process.

Company:

Companies can register on the platform but can access their dashboard only after admin approval. Once approved, companies can create and manage placement drives by providing details such as job title, required skills, experience, and salary. They can view student applications for their drives and update the application status (such as shortlisted, selected, or rejected). This allows companies to efficiently manage the recruitment process within the system.

Student:

Students can register, log in, and maintain their profiles by adding details such as skills, education, and resume. They can view only approved placement drives and apply for relevant opportunities. Students can also track the status of their applications and view their application history. The system ensures that students cannot apply multiple times to the same drive, maintaining fair usage of the platform.

3. Project Details

During the development of this project, AI tools such as ChatGPT were used in a limited capacity for assistance in understanding Flask concepts, debugging errors, improving HTML templates, and refining Bootstrap-based UI components. The tool was also used to clarify SQL queries, fix routing issues, and improve the structure and clarity of the project documentation.

The AI assistance was primarily focused on guidance during development, resolving errors, and enhancing frontend design using Bootstrap classes. The core aspects of the project, including system design, database schema creation using SQLite, application logic, routing, and implementation of all functionalities, were developed and implemented manually.

The estimated extent of AI/LLM usage in this project is approximately **20–30%**, mainly for debugging support, UI improvements, and conceptual understanding, while the majority of the implementation and logic was independently developed.

4. Technologies and Frameworks Used

Technology / Library	Purpose
----------------------	---------

Flask	Main backend web framework used to build the application and handle routing, request handling, and server-side logic
SQLite	Lightweight relational database used to store students, companies, placement drives, and application data
SQLAlchemy	ORM used to define database models and manage relationships between tables
Jinja2	Template engine used to render dynamic HTML pages in Flask
Bootstrap	Used to design responsive and clean user interfaces

5. Database Schema / ER Diagram

Tables:

Admin — Stores administrator credentials used to manage and control the placement portal.

Company — Stores company details such as company name, email, password, and approval status (pending/approved/rejected/blacklisted). Only approved companies are allowed to access the dashboard and create placement drives.

Student — Stores student information including name, email, password, role, skills, education, resume, and account status (active/blacklisted). It is used for authentication as well as profile management.

Drives — Stores placement drive details created by companies, including job title, description, required skills, experience, salary, drive status (active/closed), and approval status (pending/approved/rejected). Each drive is linked to a company.

Application — Stores records of students applying to placement drives. It includes student email, drive ID, and application status (applied/shortlisted/selected/rejected). A unique constraint ensures that a student cannot apply multiple times to the same drive.

Relationships:

Company → Drives

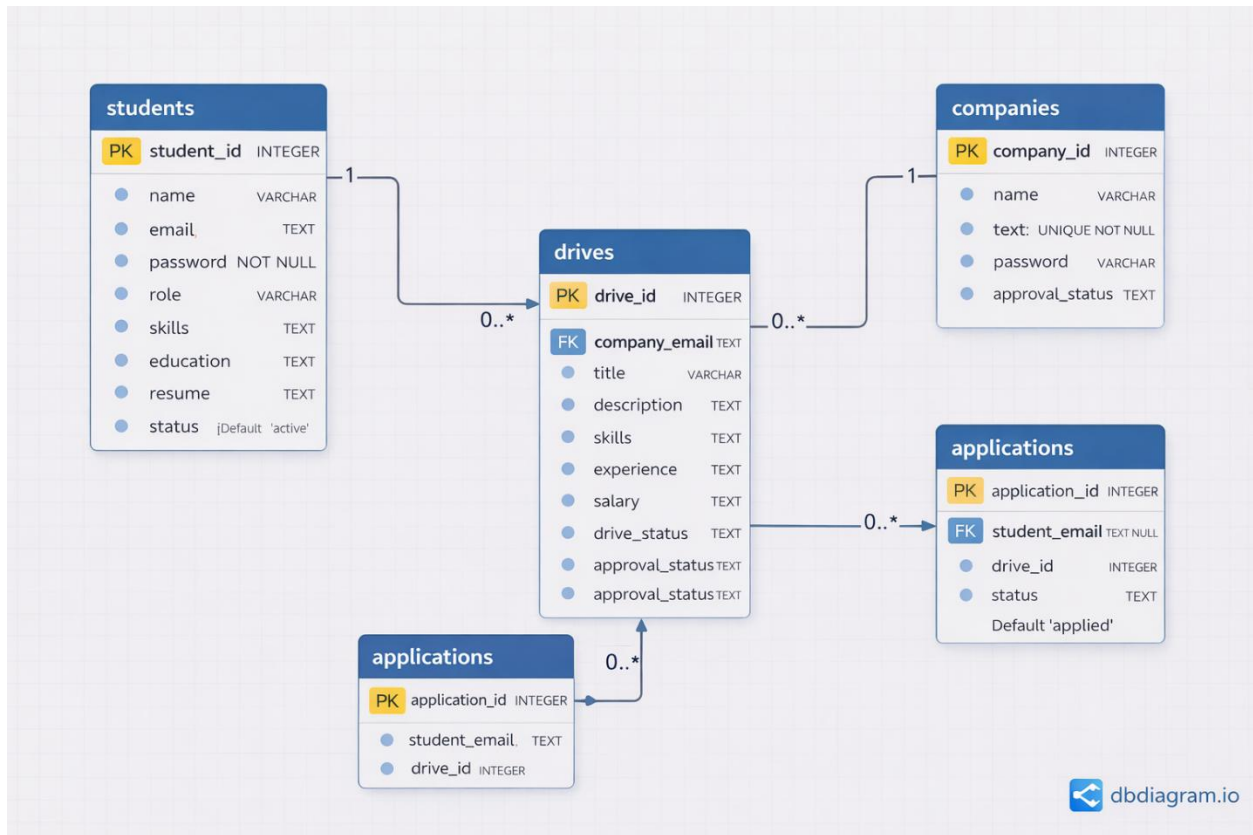
One company can create multiple placement drives, while each drive is associated with a single company through the company_email field.

Student → Application

One student can apply to multiple placement drives, and each application is linked to a specific student using student_email.

Student ↔ Drives

There exists a many-to-many relationship between students and drives, which is resolved using the **Applications** table. A unique constraint ensures that a student can apply only once to a particular drive.



6. API Resource Endpoints

No external REST API endpoints were implemented in this project.

7. Architecture and Features

Architecture Overview

The application follows a Flask-based web architecture where the backend handles routing, business logic, and database operations, while the frontend is rendered using Jinja2 templates with Bootstrap for styling. The system uses SQLite as the database, which is created programmatically using SQL queries inside the application.

app.py — This is the main entry point of the application. It initializes the Flask app, creates database tables using SQLite, and contains all route definitions, business logic, and role-based access control using session management.

Database (SQLite) — The database is managed using the sqlite3 module instead of ORM frameworks. It includes tables such as students (also storing admin via role), companies, drives, and applications, with relationships maintained using foreign keys like email and drive_id.

templates/ — Contains HTML pages rendered using Jinja2, including login, registration, dashboards (admin, company, student), and placement drive views.

8. Project Details

Drive Link:

<https://drive.google.com/file/d/1sEtDQqVIHNQemlp0xEZ4iR0fXhKc2yu/view?usp=sharing>